CLDV7112W · CLOUD DEVELOPMENT B · ADADW1

ABC Retail Storage Console

A web application built on Azure Table Storage and Azure Blob Storage

LIVE APPLICATION

https://abcretail-gm-cldv7112.azurewebsites.net

PUBLIC GITHUB REPOSITORY

https://github.com/geniustaku/abc-retail-storage-console

ACCESS

Open. The application requires no sign in, so no credentials are needed to review any page or to exercise the create, edit and delete paths.

STUDENT NAME

Genius Mhirizhonga

STUDENT NUMBER

ST10541838

SEMESTER

Semester 2, 2026

ASSESSMENT

ICE Task 1

MODULE CODE

CLDV7112w

DUE DATE

7 August 2026

SECTION 1

Brief and how it was met

ABC Retail ran order processing on ageing on-premises infrastructure. One relational database absorbed both the reads of customers browsing and the writes of orders landing, which is why it buckled under seasonal peaks, while product images sat on network shares that were slow to read and awkward to grow. This application moves both concerns onto Azure Storage services that scale independently of each other and of the web tier.

REQUIREMENT	HOW IT IS SATISFIED
Store customer profiles using Azure Tables	The `CustomerProfiles` table holds one entity per customer, with full create, read, update and delete from the site.
Store product related information using Azure Tables	The `Products` table holds one entity per catalogue item, partitioned by category, with full create, read, update and delete.
Host images and multimedia using Azure Blob Storage	Uploads are written to the `product-images` container and served to the browser directly from the storage account by blob URL.
Screenshots evidencing both services in the website	Section 6, figures 2 to 7.

SECTION 2

Azure resources and cost

RESOURCE	NAME	TIER
Resource group	`rg-abcretail-cldv7112`	South Africa North
Storage account	`stabcretailgm001`	StorageV2, Standard LRS, Hot
App Service plan	`asp-abcretail-free`	F1 Free, Linux
Web app	`abcretail-gm-cldv7112`	.NET 10, HTTPS only

Standard LRS is the cheapest replication tier and F1 is free, so the running cost of the whole deployment is the few cents of storage the data actually occupies. Every resource is created by `infra/provision.sh` in the repository, which makes the environment reproducible rather than hand-assembled.

SECTION 3

Data design

Azure Table Storage is a NoSQL key-value store. It offers no joins, no aggregate functions and no secondary indexes, so the partition key and row key carry the weight that indexes would carry in a relational database. The diagram below is therefore an entity design rather than a classical relational ERD: the two tables are independent, and the association between a product and its image is held by convention in the entity itself.

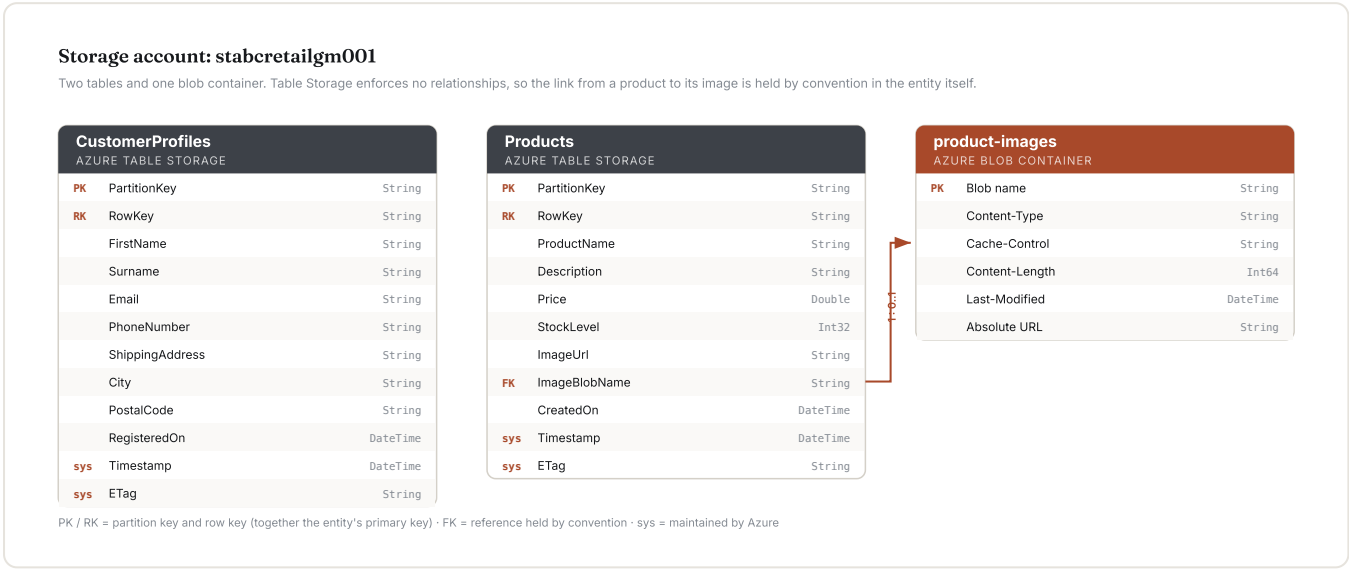

Figure 1. Entity design across the two tables and the blob container.

CustomerProfiles — partitioned under a single key

Every customer sits in the partition `CUSTOMER`, with a generated GUID as the row key. At profile volumes a single partition is comfortable, and keeping the table that way means any group of customers can still be written in one entity group transaction, since those are scoped to a partition. If the customer base outgrew one partition server, region would be the natural key to split on, because fulfilment queries are already regional.

Products — partitioned by category

The catalogue is browsed one category at a time far more often than it is read whole. Using category as the partition key turns that dominant query into a single-partition scan instead of a full table scan, and spreads writes across partition servers during a bulk catalogue load. The cost is that the partition key forms part of an entity's identity, so moving a product between categories is a delete followed by an insert rather than an update in place. The edit path handles that case explicitly.

Prices are stored as `Double` because Table Storage has no decimal type.

Why images are not in the table

Table Storage caps an entity at one megabyte and a single property at sixty-four kilobytes, so imagery belongs in Blob Storage regardless of preference. The product entity holds two fields instead: `ImageUrl`, so a view can point an image tag straight at the blob, and `ImageBlobName`, so the blob can be replaced or deleted when the product changes.

SECTION 4

Implementation

The application is ASP.NET Core MVC on .NET 10. Both storage services sit behind interfaces resolved through dependency injection, so neither the controllers nor the views hold a reference to an Azure SDK type.

COMPONENT	RESPONSIBILITY
`TableStorageService<T>`	Generic persistence over one Azure Table using `Azure.Data.Tables`.
`BlobStorageService`	Upload, delete and count blobs using `Azure.Storage.Blobs`, with content type and size validation before anything reaches the network.
`CustomersController`	Create, read, update and delete against the CustomerProfiles table.
`ProductsController`	Create, read, update and delete spanning both the Products table and the blob container.

Client lifetime

Both clients are registered as singletons. `TableClient` and `BlobContainerClient` are thread safe and pool their connections, so one instance for the lifetime of the application avoids the socket exhaustion that follows from constructing a client per request.

Ordering of writes across the two services

Creating a product uploads the blob before writing the table entity, and deleting one removes the blob before deleting the entity. A failure partway through therefore leaves an unreferenced blob rather than a catalogue entry pointing at an image that does not exist, which is the cheaper of the two failures to reconcile.

Serving images

The container grants anonymous read at blob level, so the browser fetches each image directly from the storage account rather than routing bytes through the web application. That keeps thumbnail traffic off the App Service compute budget entirely. Where the imagery was not already public, shared access signatures would be the alternative, at the cost of a signing round trip per image.

SECTION 5

Securing the storage key

The repository is public, so the storage account key appears nowhere in it. This was verified against the working tree before the first commit.

- **Locally** the key is held in .NET user secrets, which live outside the project directory entirely and therefore cannot be committed by accident.
- **In Azure** it is an App Service application setting named `ConnectionStrings__AzureStorage`. The configuration provider reads a double underscore as a colon, so the application resolves it as `ConnectionStrings:AzureStorage` with no code difference between the two environments.
- `appsettings.json` carries the key with an empty value, purely to document that a value is expected.
- **Deployment** authenticates with a publish profile held as an encrypted GitHub Actions secret. GitHub withholds secrets from fork pull requests, so a public repository does not expose it. The storage key is not involved in deployment at all.

> Because the site holds no credentials and requires no sign in, a marker can open any page and exercise every create, edit and delete path without a username or password.

SECTION 6

Screenshots

All captures are of the deployed application at https://abcretail-gm-cldv7112.azurewebsites.net, reading and writing the live `stabcretailgm001` storage account.

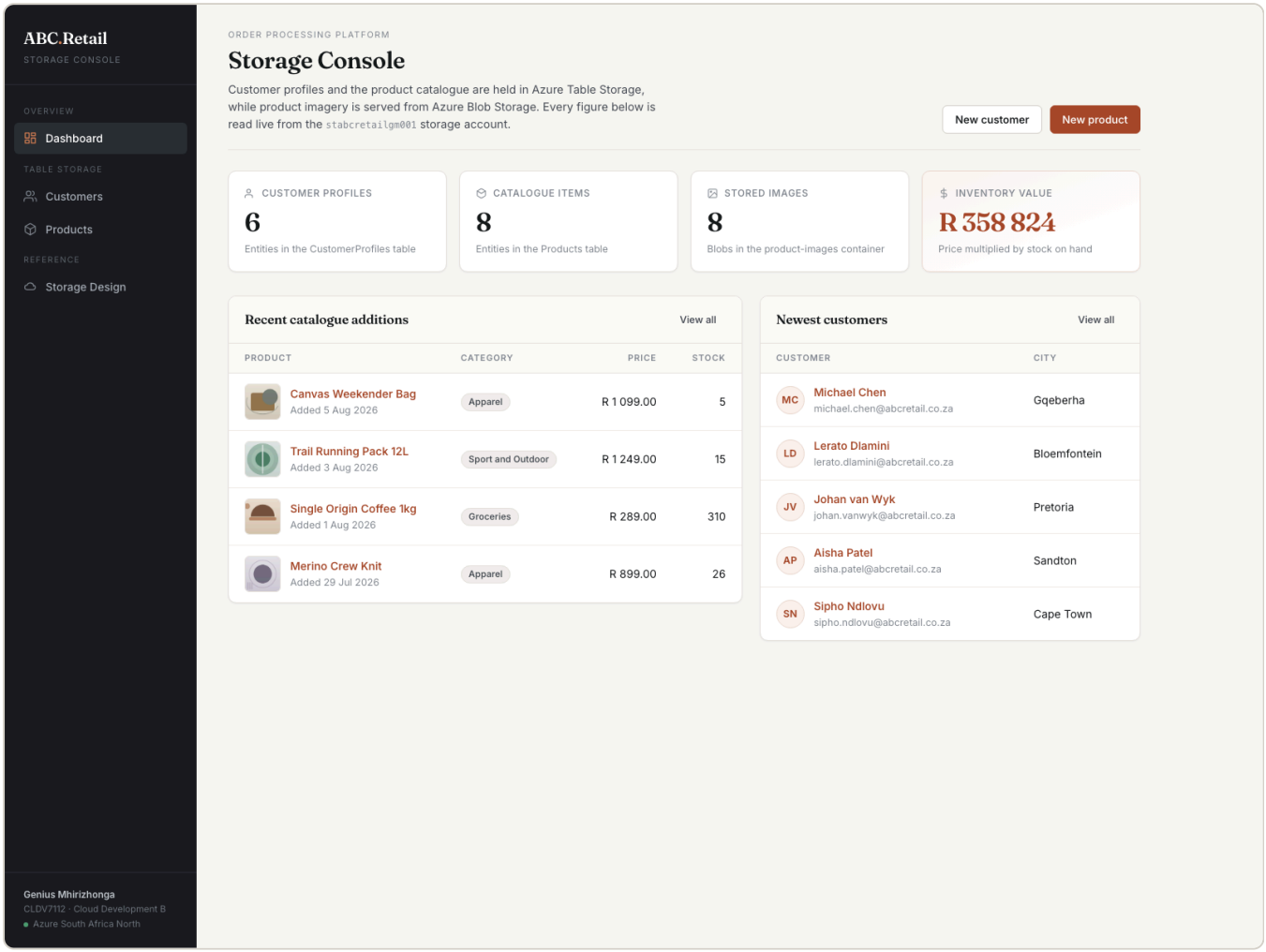

Figure 2. Dashboard. Entity and blob counts read live from the storage account on every request. Six customer entities, eight product entities and eight blobs, with inventory value computed across the catalogue.

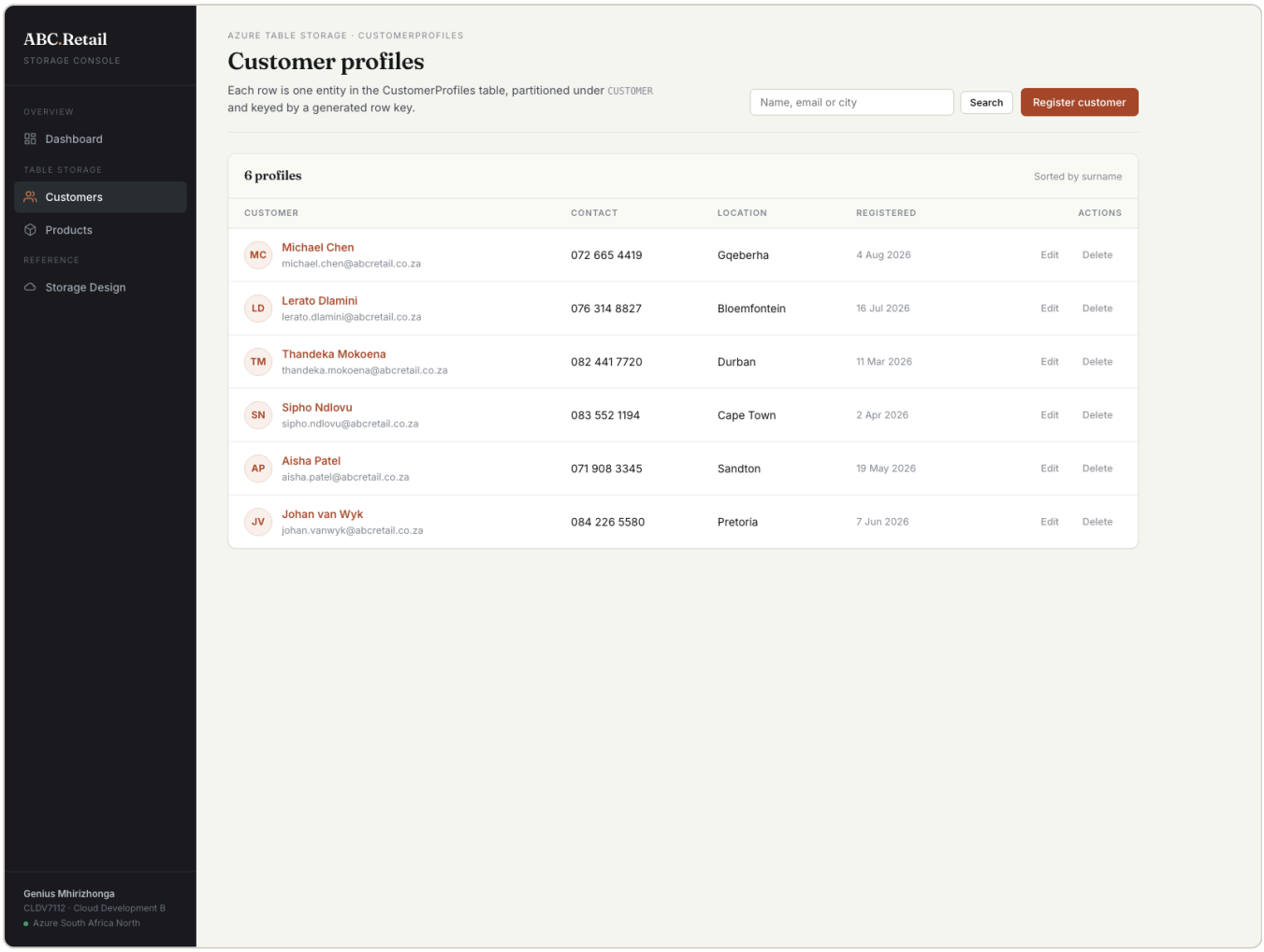

Figure 3. Customer profiles from Azure Table Storage. Every row is one entity in the CustomerProfiles table, partitioned under CUSTOMER and keyed by a generated row key.

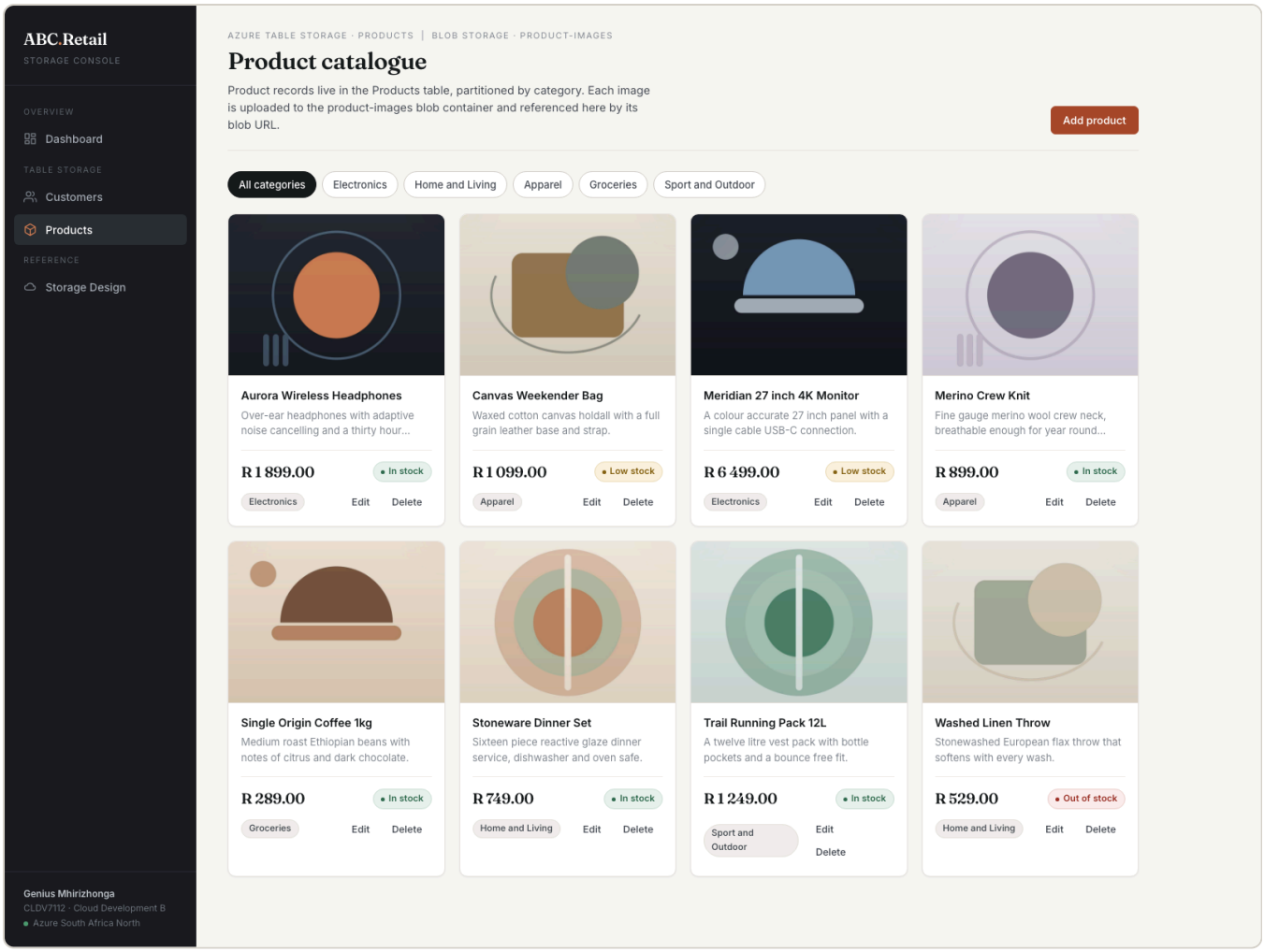

Figure 4. Product catalogue with Blob Storage imagery. Product records come from the Products table; every image on this page is fetched by the browser directly from the product-images blob container, not proxied through the web application.

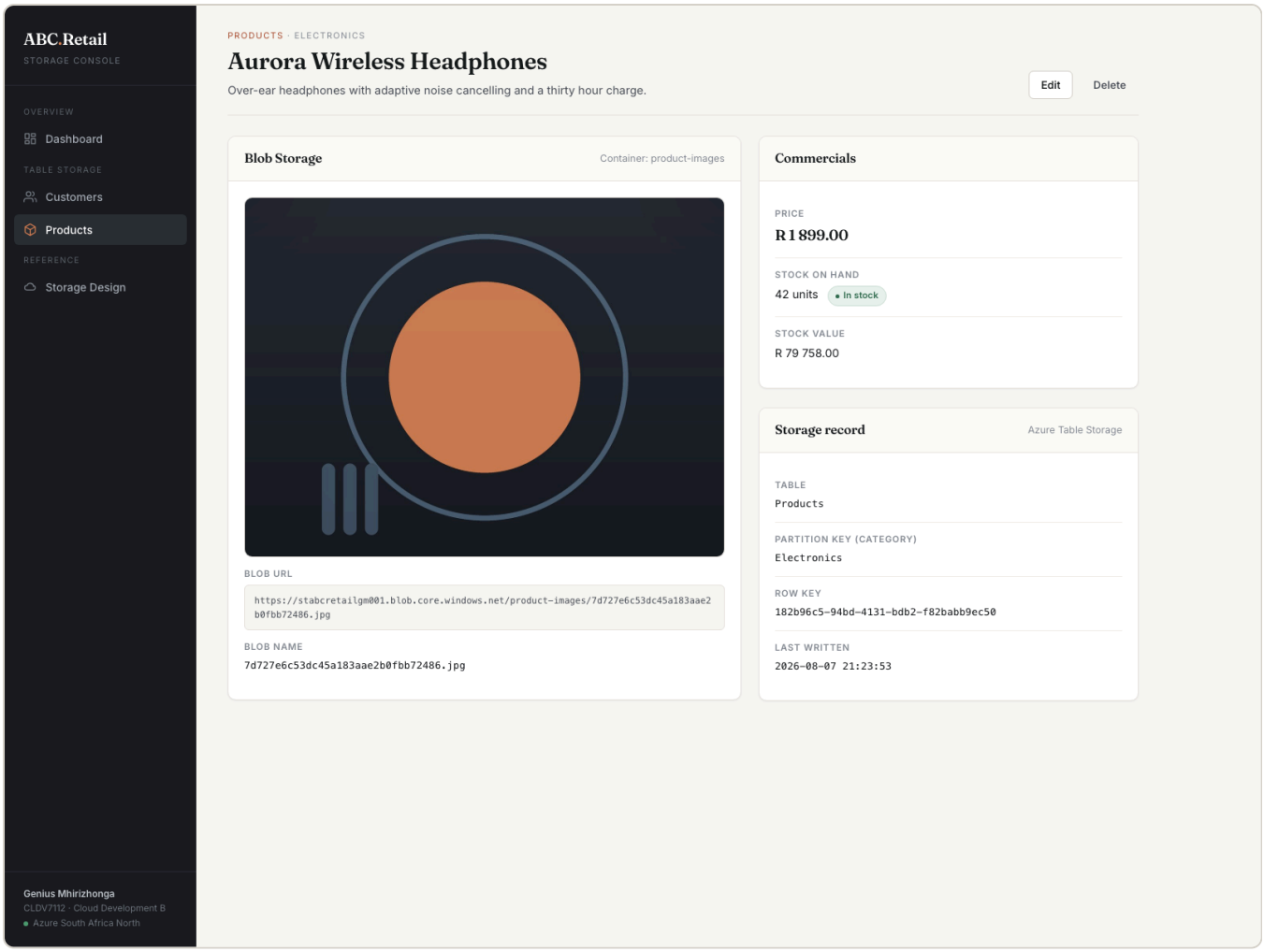

Figure 5. Both services on one record. The clearest single piece of evidence. On the left, the blob URL and blob name held in Blob Storage. On the right, the table, partition key, row key and last-written timestamp held in Table Storage.

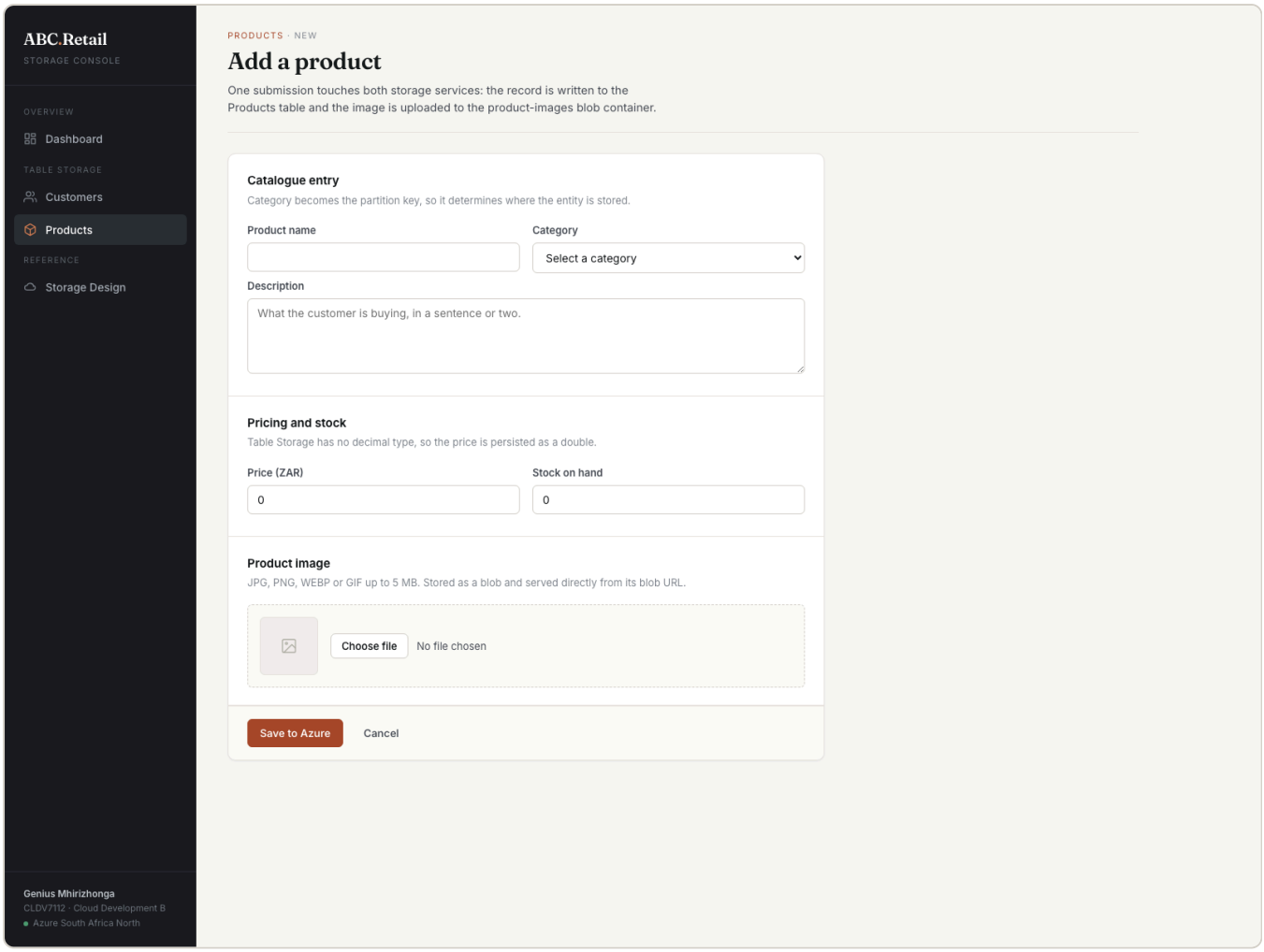

Figure 6. The write path. One submission touches both services: the image is uploaded to the blob container first, then the entity is written to the Products table carrying the resulting blob URL.

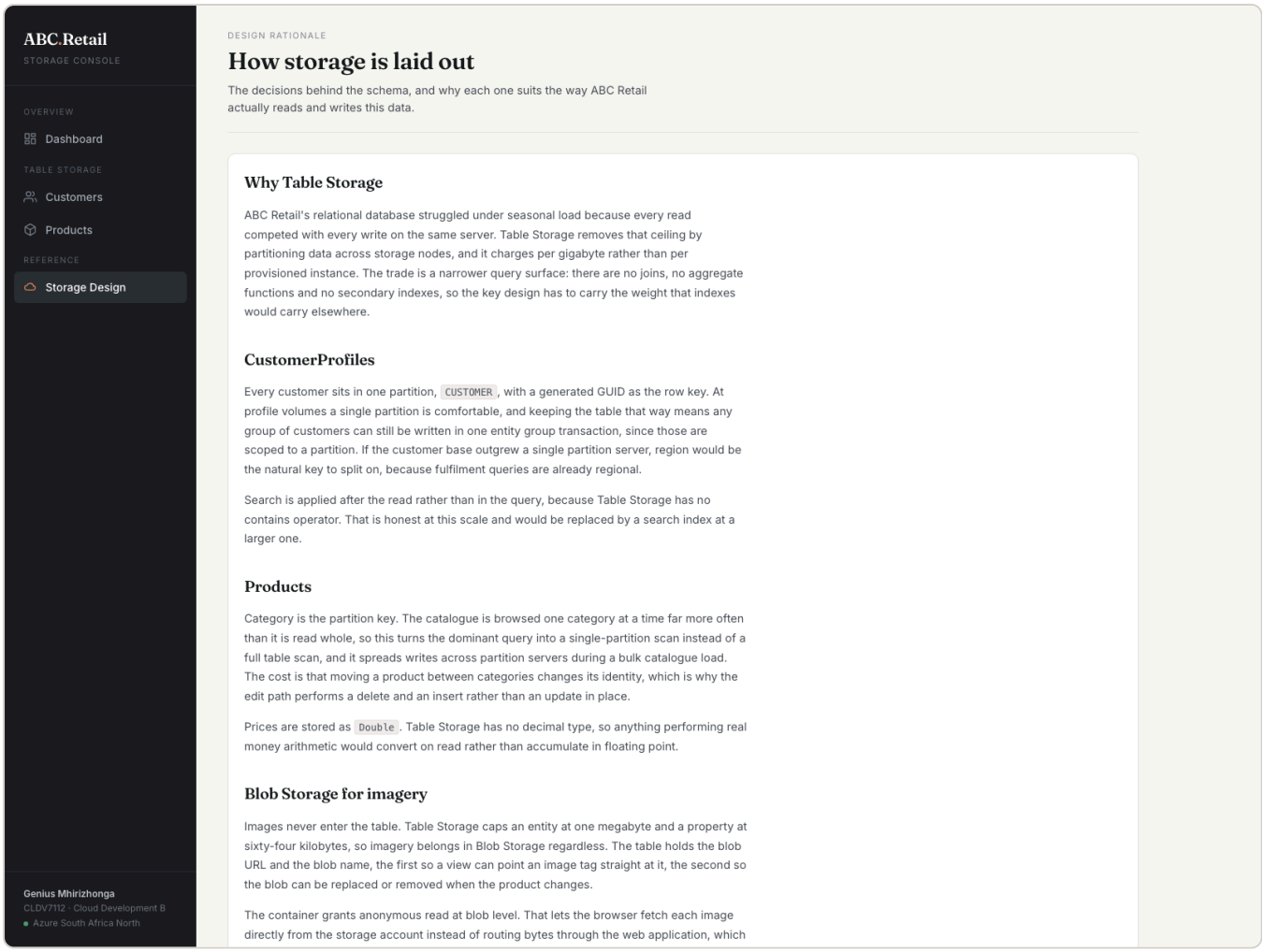

Figure 7. Design rationale published in the application. The partition key and container decisions are documented in the running site itself.

SECTION 7

Verification

The create and delete paths were exercised against the live storage account rather than assumed. Adding a product wrote its entity to the Products table and raised the blob count in the container from eight to nine; deleting the same product removed both, returning the count to eight. That testing also surfaced a defect worth recording: a number input always posts its value in invariant form, so on a host whose locale separates decimals with a comma a valid price such as `439.50` failed to bind and the form rejected it. The application now pins itself to the invariant culture and formats amounts for display through a dedicated helper, which makes a local run and an App Service instance behave identically.

SECTION 8

Where this goes next

The brief also describes unreliable message queuing and a struggling analytics pipeline. Because the storage services already sit behind interfaces resolved through dependency injection, adding an

Azure Queue Storage service or an Azure Files share is additive rather than a rewrite: a new implementation is registered alongside the existing two and injected where it is needed, with no change to the controllers or views that exist today.